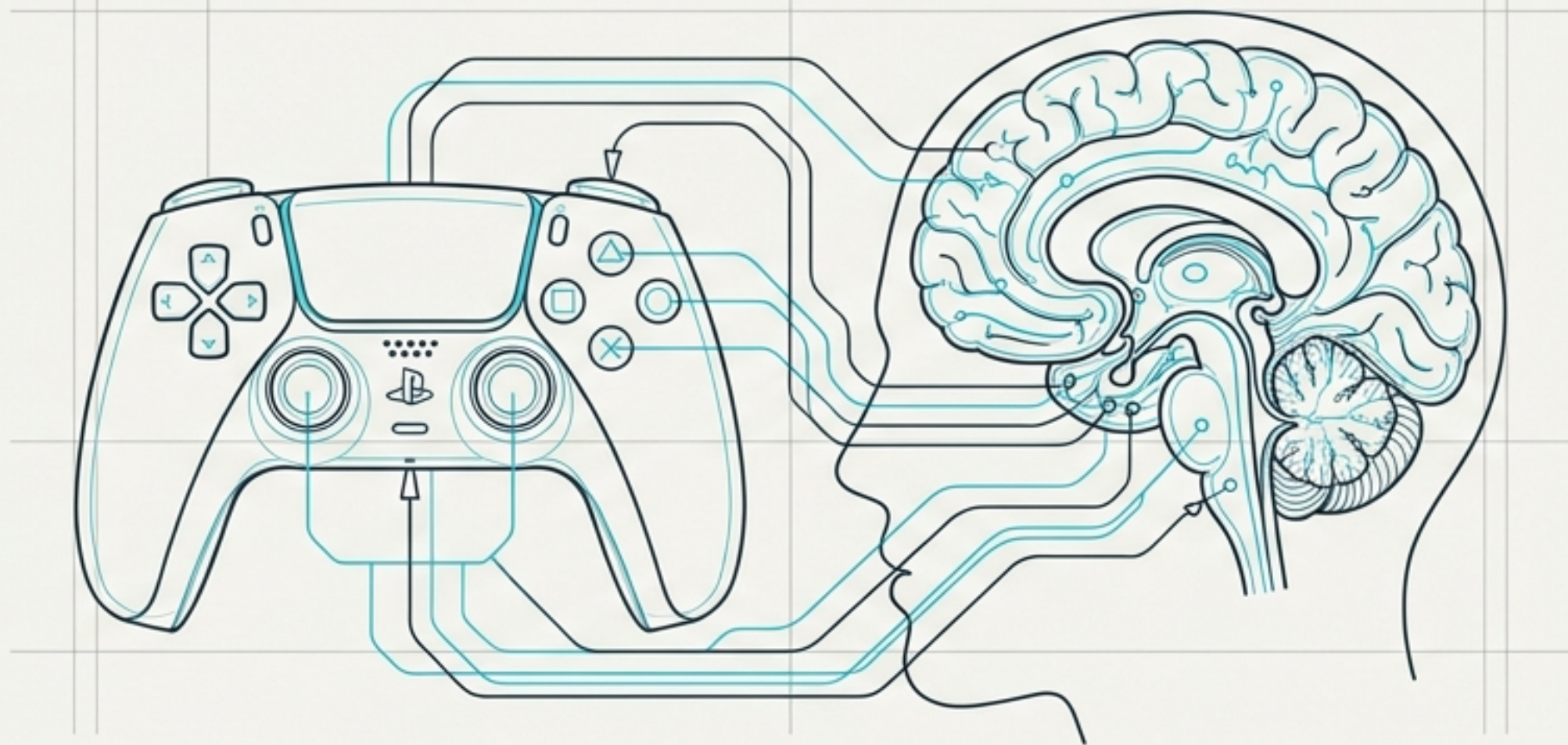




犯错即游戏：人类失误视角的启发

面向游戏设计师的 SRK 认知模型与容错设计指南

[Based on insights from Epic Games UX Research]

重新定义“失误”



核心法则

UX 优化的目的绝不是让游戏变得简单，而是确保玩家在对
对抗游戏本身，**而不是在对抗
界面的设计缺陷。**



无意失误 (UX 摩擦)

在工业界，所有失误都是致命的，必须完全消除。
在游戏中，这类失误代表玩家与交互界面或
糟糕引导的无谓对抗。

结论：必须绝对预防。

有意失误 (游戏性挑战)

游戏设计的核心是合理的挑战。
这类失误代表玩家在与游戏机制进行有意义的
博弈。

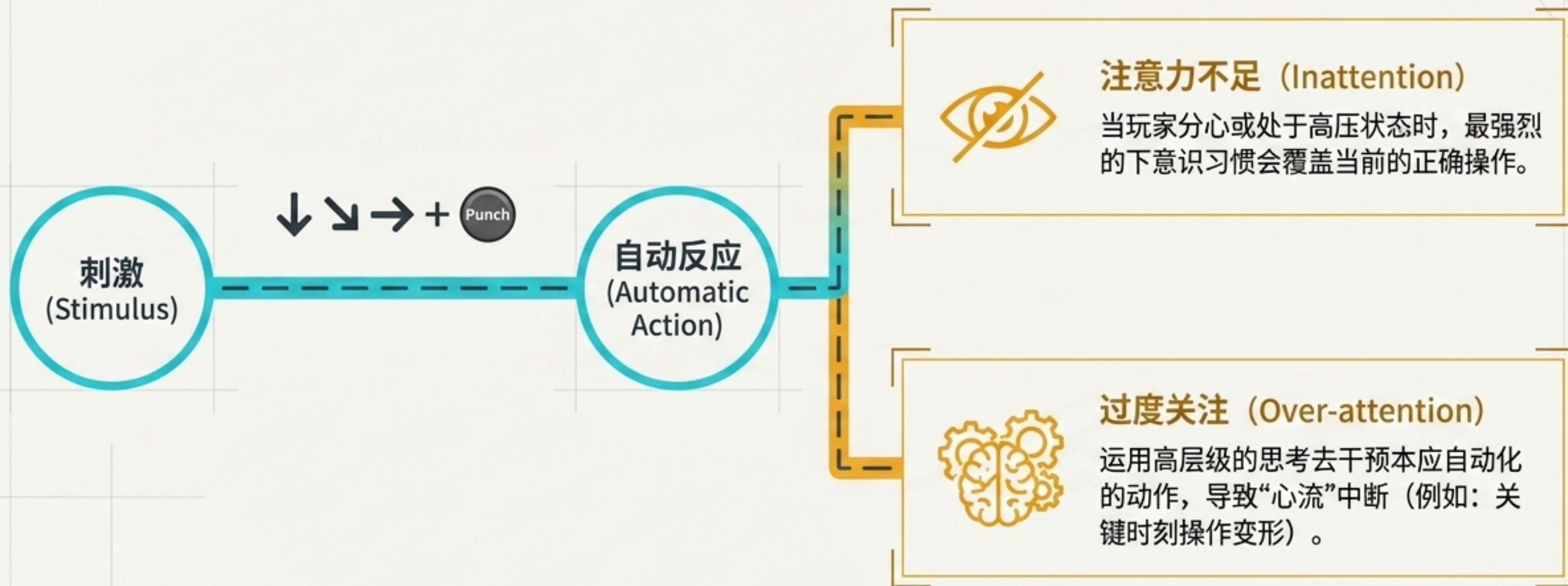
结论：必须精心鼓励。

认知蓝图：SRK 模型



人类大脑本质上是“懒惰”的，总是尽可能将认知降级到最低负荷的技能层。

Level 1: 技能层失误 - 肌肉记忆的背叛



诊断：双重捕获失误 (Double Capture Slips)

强习惯在压力下覆盖当前需求

UX 预防：标准化与一致性



现象：玩家遇到惊吓想按键**蹲下**隐藏，却因常驻习惯按下了**建筑模式**按键，导致暴露。

方案：**高度标准化控制方案**。确保在紧急情况下，玩家最强烈的**下意识反应**就是正确操作。

设计利用：关键时刻打破自动驾驶



现象：Boss战中，玩家突然被**倒吊**，操作方向**完全反转**。

方案：故意**颠倒按键**，强行将玩家大脑从“**肌肉记忆**”状态踢出，制造极致的**紧张感**与**挑战**。

诊断：知觉混淆 (Perceptual Confusion)

视觉负荷：“这个东西看起来像那个东西”

UX 侧：必须预防



创造极具**辨识度的轮廓** (Silhouettes)、**清晰的动画反馈**以及 **HUD 高光提示**。防止玩家因视觉噪点误判目标位置或阵营。

游戏设计侧：主动利用



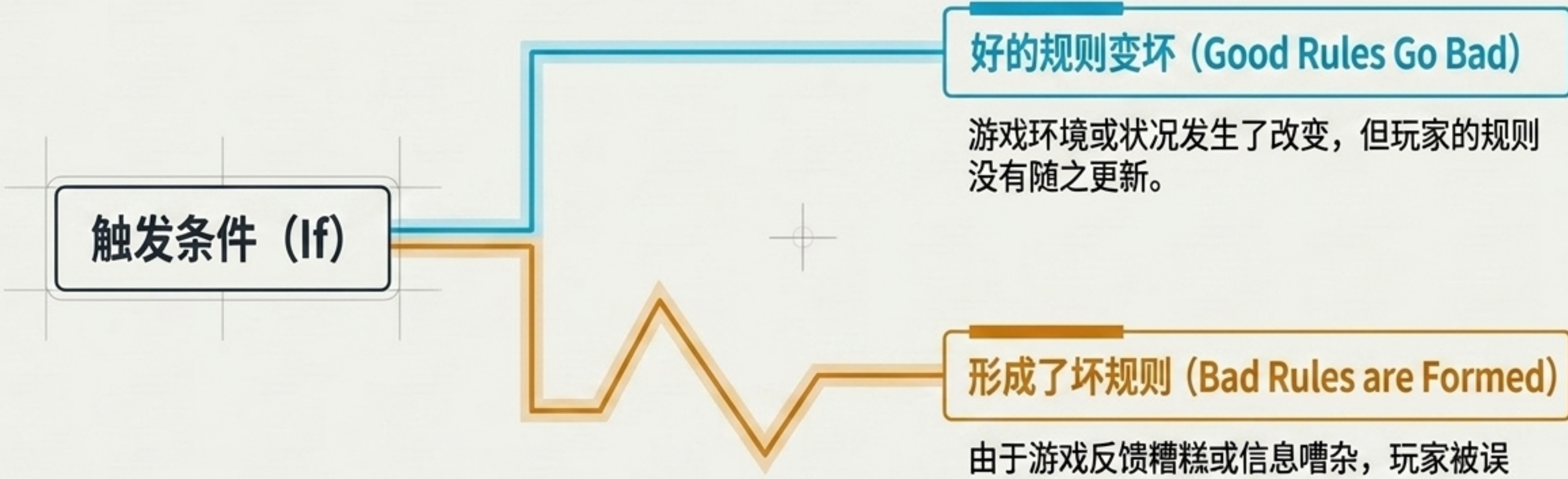
游戏设计侧：主动利用

故意增加视觉负荷来创造玩法深度。例如：找茬游戏、伪装成宝箱的怪物 (Mimics)、隐形敌人、以及间谍/埋伏机制。

Level 2: 规则层失误 - 条件反射的陷阱

玩家开始依赖后天学习的启发式规则（Heuristics）推进游戏。此时的失误通常源于规则的错位。

触发条件 (If)



好的规则变坏 (Good Rules Go Bad)

游戏环境或状况发生了改变，但玩家的规则没有随之更新。

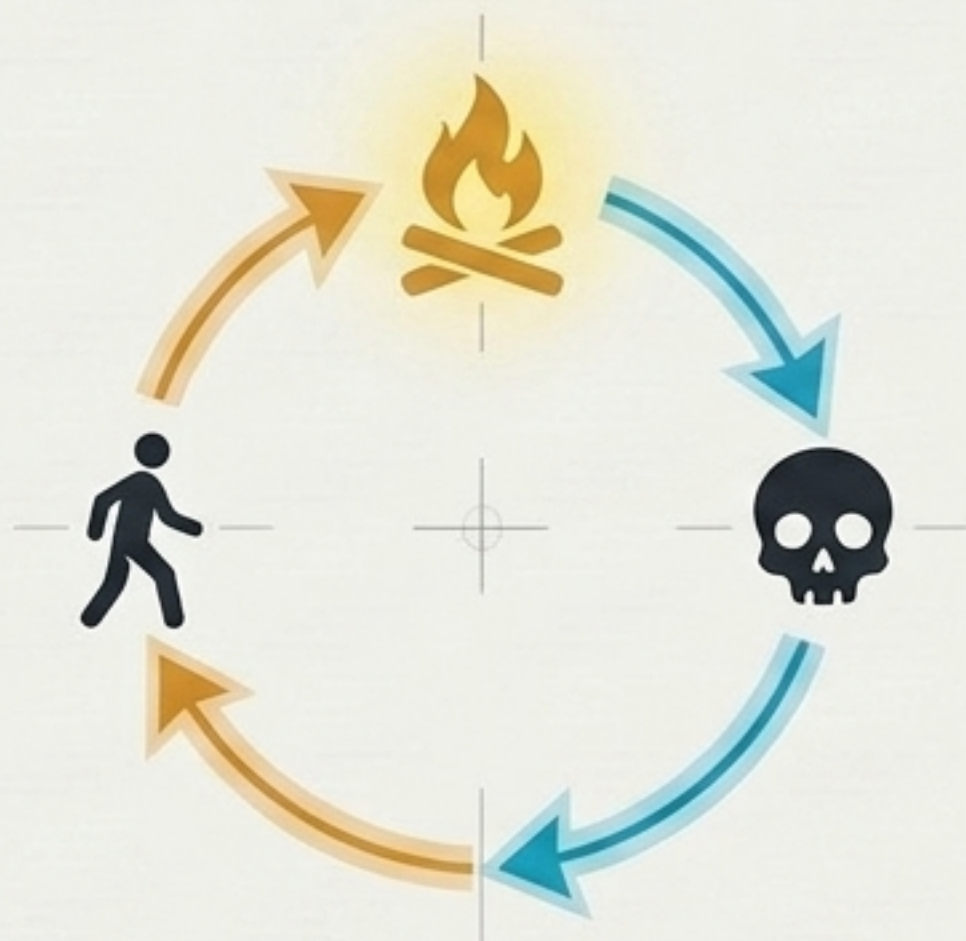
形成了坏规则 (Bad Rules are Formed)

由于游戏反馈糟糕或信息嘈杂，玩家被误导，学会了一种类似“迷信”的错误规则。

诊断：首次例外与策略固化

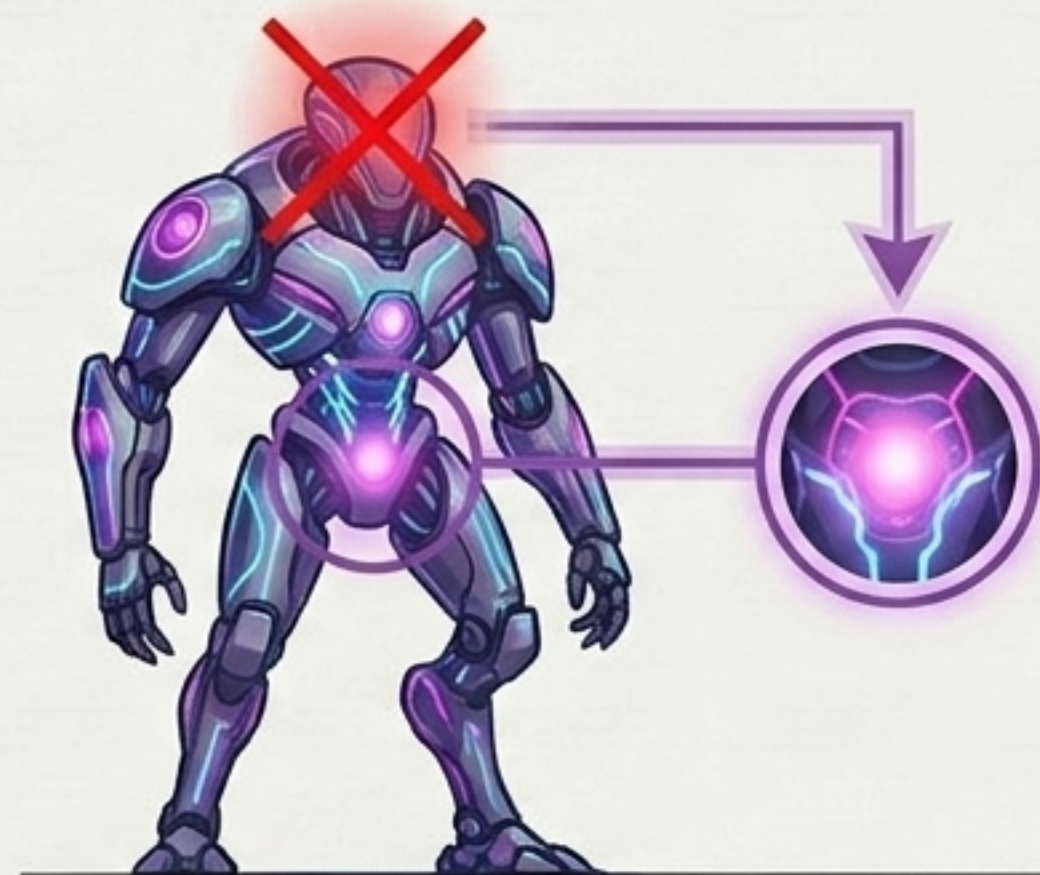
玩家极度依赖主导策略（“没坏就别修”）

UX 预防：极致的一致性



如果策略失效，必须提供压倒性的 UI 反馈。例如：当防御塔对魔法免疫时，单纯取消伤害数字是不够的。必须弹出巨大的“免疫”视觉提示，打破玩家的固化认知。

设计利用：引入例外逼迫换阵



故意引入特定免疫机制强迫玩家切换流派。例如：将敌人的弱点从常规的头部移至腹部，打破玩家的射击肌肉记忆，增加动态博弈的乐趣。

诊断：错误编码与玩家迷信 (Encoding Deficiencies)

当反馈嘈杂时，玩家会自行发明“假规则”

渐进式解锁的 UX 重构 (Progressive Unlocking)

1. 最初的迷信

玩家错误地将“击中后出现的蓝圈”视为普通的“打击标记”，而非“加速采集的弱点”。疯狂堆叠 UI 视觉特效也无法纠正这种迷信。



2. 剥夺与明确解锁

初始剥夺该能力。随后通过技能树明确解锁该工具，让玩家建立正确的心理模型。



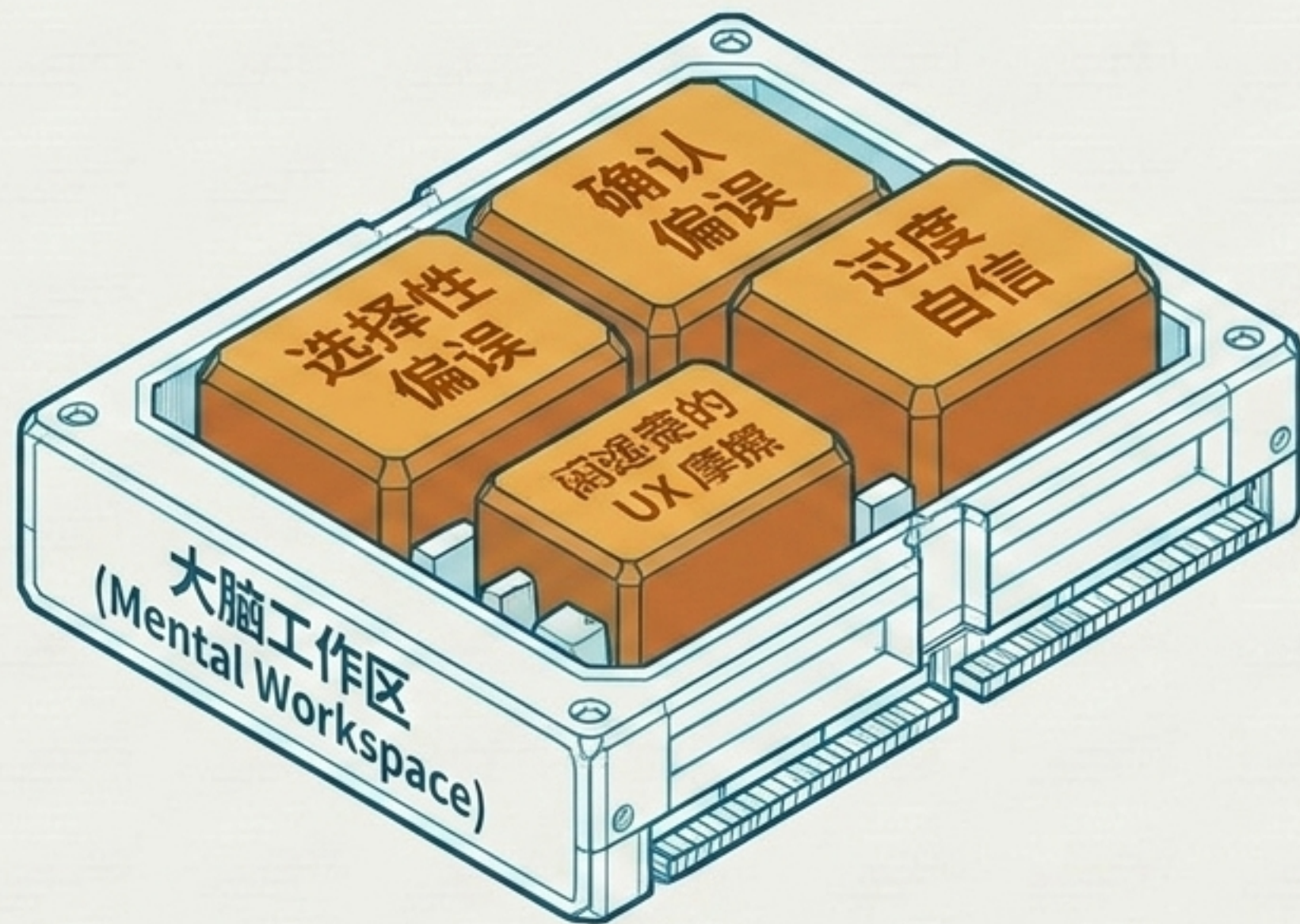
3. 强制实践

强制玩家成功使用一次该机制后才允许继续推进。彻底解决认知编码错误。



Level 3: 知识层失误 - 认知偏差与大脑过载

最高层级的逻辑处理效率极低，极易受到大脑“带宽”限制的影响。



核心认知：玩家在面临复杂选择时，通常不会进行绝对理性的“最优解”计算。相反，他们会受制于认知偏差，选择那些在心理上最显著（Salient）或最容易获取（Available）的方案。

诊断：选择性聚焦与确认偏误



选择性聚焦 (Selectivity)

现象：玩家优先选择情绪上最具吸引力的行为（如死磕收益极低的“战利品洞穴”），而非逻辑上最高效的路径。

UX 方案：让逻辑上正确的选择，在视觉和听觉上发光发亮，将理性选择与感性吸引力对齐。



确认偏误 (Confirmation Bias)

现象：玩家固守预期结果（即便有新地点，依然无脑跳往老城区）。

设计利用：故意利用预期制造谜题。布置视觉极其复杂的“红鲱鱼”（误导线索），而实际解法极其简单，利用玩家的惯性思维制造顿悟感。

诊断：复杂性与过度自信

人类不擅长延迟反馈，但极度盲目自信

~~Base + 20% * 1.5x~~ ✗



应对复杂性

人类大脑极度不擅长处理“延迟反馈”和“指数级数学”。

UX 方案：拒绝让玩家做算术题。采用即时、线性的 UI 可视化设计直观展示数据差异。

拥抱过度自信

玩家常常无视警告标志，高估自身能力。

黄金法则：这正是玩家持续玩游戏的原因！利用玩家的盲目自信，设计巨大的 Boss 和陡峭的难度曲线，将挫败转化为强大的参与驱动力。

SYNTHESIS: SRK 游戏设计诊断矩阵

系统级容错与玩法设计速查表

认知层级 (Level)	玩家心态 (Mindset)	常见失误 (Error Type)	UX预防 (Prevent)	关卡利用 (Exploit)
技能层 (Skill)	自动驾驶/肌肉记忆	注意力不足/双重捕获	统一操作标准	关键时刻反转控制
规则层 (Rule)	如果-那么/条件反射	首次例外/策略固化	绝对一致性/巨型UI反馈	引入免疫机制逼迫换阵
知识层 (Knowledge)	缓慢思考/逻辑计算	确认偏误/过度自信	让正解视觉高光/ 线性反馈	视觉欺骗/利用自信 制造Boss战

宏观视角：瑞士奶酪模型 (Swiss Cheese Model)

从个体心理到系统设计

失误极少是由单一故障点（或某个“愚蠢的玩家”）直接导致的。

隐性失效 (Latent Failures)

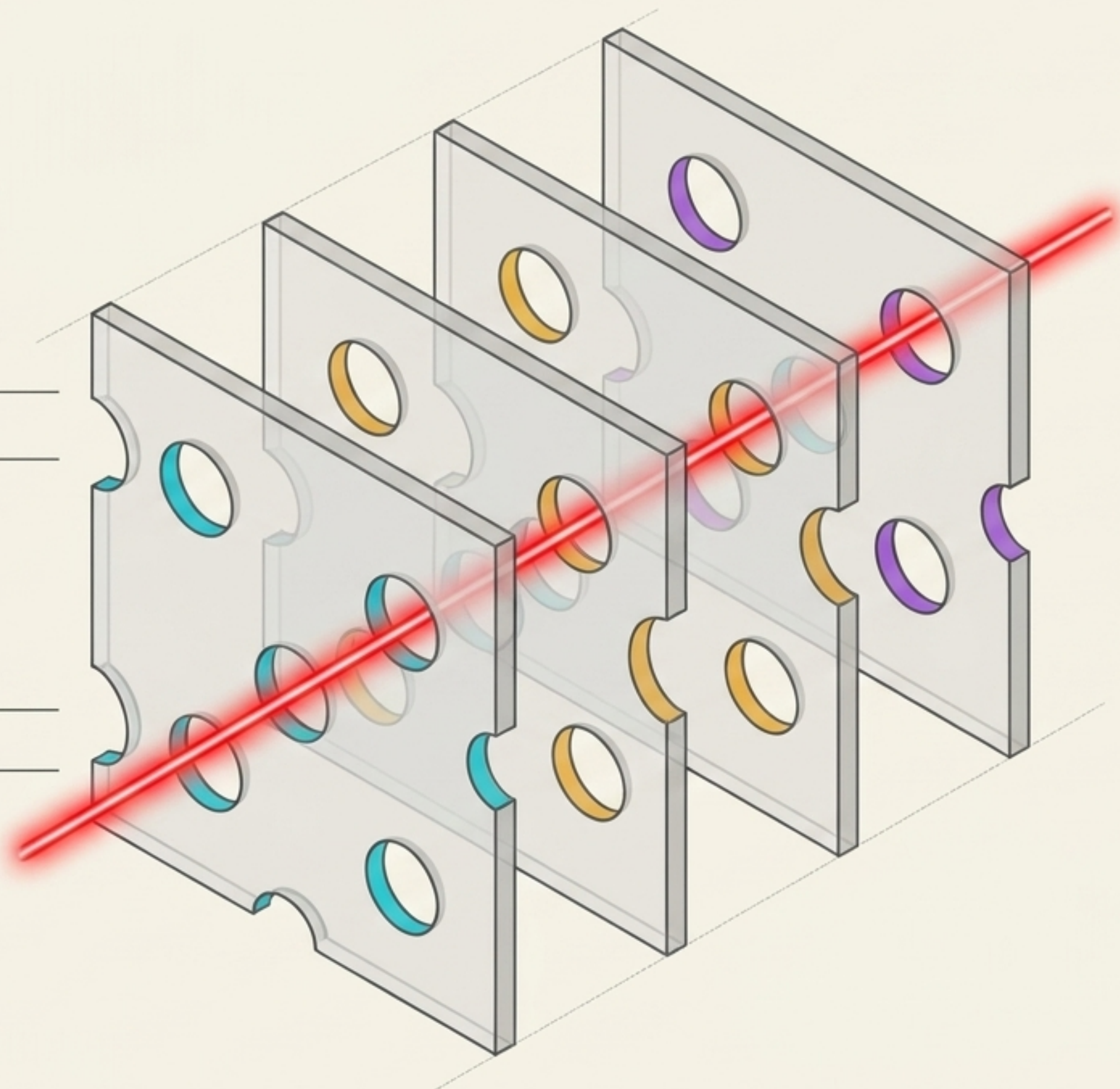
每一片奶酪代表游戏的一层系统（如：[引导系统](#)、[UI 对比度](#)、关卡灯光、机制设计）。

致命对齐

只有当多个重叠系统中的“漏洞”在同一时刻完美对齐时（如红线所示），致命的玩家失误才会发生。

核心结论

不要孤立地修补玩家行为，要审查整个系统堆栈。



建立安全文化：从玩家到团队

责备文化 (Blame Culture)

聚焦于寻找责任人。

对于玩家：在 UX 报告里写“玩家感到困惑，未按预期操作。”

对于团队：将严重的加班或糟糕的代码提交归咎于单纯的个人错误。

结果：停滞创新，隐患不断重演。

安全文化 (Safety Culture)

聚焦于系统性修复。

对于玩家：在报告里写“蓝色文本缺乏对比度，导致玩家错失目标。”——修复系统，而非指责玩家。

对于团队：认识到工作失误是由于排期、工具链或沟通流程的系统性崩溃。

结果：系统思维驱动真正的优化。

拥抱有意的挑战 · 消灭无意的摩擦 · 构建强健的系统